Projet Web Sémantique:
Extraction de graphes de connaissances à partir de texte

<u>Réalisé par:</u>

Jennifer SAID

Nour CHERIF

Lina CHENNI

<u>Enseignant:</u>

Zoubida KEDAD

CurEx : un système pour extraire, enrichir et explorer des graphes de connaissances spécifiques à un domaine à partir de textes

L'objectif principal de CurEx est d'intégrer des données structurées (Wikidata, DBpedia, …) et non structurées (articles de journaux, rapports annuels, …) dans une base de connaissances unifiée et spécifique à un domaine, afin de centraliser les données et permettre leur exploitation sous forme de graphe.

La création automatique d'une telle base implique plusieurs défis:

- Eviter les doublons afin d'identifier de manière unique chaque entité
- Résoudre les entités (entity resolution) pour fusionner celles qui se réfèrent au même objet du monde réel
- Extraire et intégrer des informations issues de textes non structurés.

Ces tâches sont complexes et nécessitent généralement un effort manuel important. CurEx propose donc un système permettant d'automatiser une grande partie de ce processus.

Le système final contient plus de 2,1 millions d'entités et environ 18 millions de relations.

CurEx repose sur une architecture modulaire : chaque composant traite un sous-problème spécifique et peut être remplacé ou amélioré indépendamment.
Il est basé sur Apache Spark et Cassandra, ce qui lui permet de traiter de grandes quantités de données.

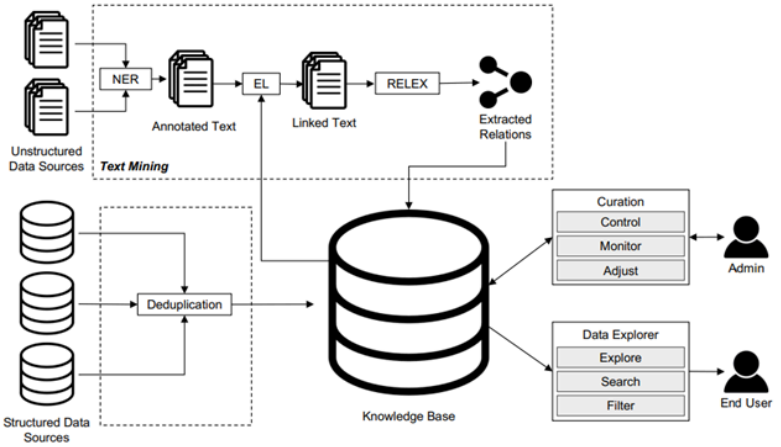


Figure 1: System Architecture

CurEx est divisé en deux grands composants : Intégration (sources de données structurées) et Text Mining (extraction d'informations à partir de sources données non structurées et intégration ultérieure dans la base).

1. Intégration

L'objectif du composant d'Intégration est d'intégrer plusieurs sources de données structurées, en fusionnant toutes les entités qui se réfèrent au même objet réel. Ce composant est divisé en deux sous-parties :

-<u>Normalisation</u> :

Les attributs sémantiquement identiques mais nommés différemment selon les sources sont uniformisés et fusionnés si nécessaire.

Exemple : "coordinate_location" et "geo.coords" deviennent un même attribut.

-<u>Déduplication</u> : On compare les entités pour éviter les doublons.

La déduplication permet d'identifier et fusionner les entités provenant de différentes sources mais représentant le même objet réel.

Pour éviter une comparaison trop coûteuse entre toutes les entités (comparaison quadratique), les auteurs utilisent une technique appelée blocking : les entités sont regroupées selon une clé commune (par exemple le code postal) et les comparaisons sont réalisées uniquement à l'intérieur de ces blocs.

Ensuite, plusieurs attributs (nom, identifiant, URL, …) sont comparés à l'aide de différentes mesures de similarité. Les scores sont combinés par une combinaison linéaire, puis normalisés dans l'intervalle [0,1]. Si le score dépasse un seuil défini, les

entités sont fusionnées.

Résultats après déduplication: 2,1 millions d'entités finales, 29 000 entités présentes dans exactement 2 sources, 3 700 présentes dans 3 sources.

2. Text Mining

Après la construction de la base initiale, les auteurs l'enrichissent à partir de textes non structurés. Ce composant comprend trois sous-modules : Reconnaissance d'Entités Nommées (NER), Entity Linking (EL) et Extraction de Relations (RELEX)

Étape 1 : Reconnaissance d'Entités Nommées (NER)

Le module NER identifie automatiquement dans les textes les entités pertinentes (entreprises, organisations, …) en utilisant un modèle statistique supervisé (CRF) entraîné sur des données annotées (provenant de dictionnaires externes) afin de détecter ces entités. Cette étape permet de transformer un texte brut en éléments identifiés et exploitables.

Étape 2 : Entity Linking (EL)

Le module EL associe chaque entité détectée à l'entrée correspondante dans la base de connaissances. Cette étape est essentielle car un même nom peut désigner plusieurs entités différentes.

Le système utilise une méthode de fuzzy matching basée sur la similarité des chaînes de caractères ainsi qu'une approche plus avancée (CohEEL) qui exploite le contexte textuel pour désambiguïser les entités. Cette étape garantit que les relations extraites sont reliées aux bonnes entités dans le graphe de connaissances.

Etape 3 : Extraction de Relations (RELEX)

Ce module identifie les relations entre les entités présentes dans le texte. Dans la version actuelle, les relations sont basées sur la cooccurrence : si deux entités apparaissent dans la même phrase, un lien est créé entre elles.

Cette méthode ne précise pas toujours la nature exacte de la relation, mais elle permet d'enrichir efficacement le graphe à grande échelle. Grâce à l'architecture modulaire, il serait possible d'intégrer ultérieurement des méthodes plus avancées (par exemple des modèles neuronaux).

La base de connaissances enrichie est ensuite utilisée pour générer un graphe d'entités et de relations. Ce graphe est exporté vers une base de données orientée graphe comme Neo4j afin de permettre son exploration et son analyse.

3. Interfaces utilisateur

Les auteurs proposent deux interfaces distinctes adaptées à deux profils :

Interface de Curation (pour les ingénieurs de données)

Elle permet:

- de contrôler le processus d'intégration,
- inspecter les doublons,
- modifier directement des entités,
- gérer les versions de la base,
- d'évaluer les performances (précision, rappel, F1).

Cette interface vise à améliorer la qualité et la fiabilité des données.

Entity Landscape Explorer (ELEX) pour les utilisateurs finaux

Cette interface permet:

- d'explorer visuellement le graphe,
- rechercher des entités,
- filtrer les relations
- inspecter les attributs,
- de signaler des erreurs.

Elle facilite l'analyse des relations entre entreprises ou organisations.

4. Avantages de l'approche

- Intégration de données structurées et non structurées
- Architecture modulaire permettant des améliorations futures
- Scalabilité grâce à Apache Spark
- Double interface adaptée aux besoins techniques et utilisateurs
- Automatisation partielle de tâches complexes (déduplication, extraction)

5. Limites de l'approche

- La relation basée sur la cooccurrence reste peu précise
- Accumulation possible d'erreurs (NER → EL → RELEX)
- Dépendance à la qualité des sources de données
- Paramétrage sensible (seuils, similarités)
- Nécessité d'une supervision humaine pour la curation

Knowledge Graph Construction from Unstructured Text with Applications to Fact Verification and Beyond

L'objectif principal de l'approche proposée dans cet article est de construire un graphe de connaissances à partir d'un corpus de textes non structurés, de manière scalable et entièrement automatisée.

Les auteurs proposent une plateforme open-source appelée dstlr, capable de transformer une grande collection de documents en un graphe exploitable, puis d'enrichir ce graphe à l'aide d'un graphe de connaissances externe (Wikidata).

L'objectif n'est pas uniquement l'extraction d'entités et de relations, mais la mise en place d'un système complet couvrant tout le cycle de vie des données :

- Ingestion des documents (collecte+ intégration)
- Extraction des mentions et relations
- Liaison avec un graphe externe
- Stockage dans une base orientée graphe
- Exploitation via requêtes pour des applications comme la vérification de faits

Le système est conçu pour fonctionner à grande échelle grâce à l'intégration d'Apache Spark, Solr et Neo4j.

Technologie	Rôle dans la pipeline	Utilité
Apache Solr/Anserini	Stockage et indexation des donnée	Utilisation d'Anserini pour l'indexation et Solr pour la recherche plein texte et la gestion des flux incrémentaux.
Apache Spark	Couche d'exécution distribuée	Orchestration via Spark DataFrames pour permettre la scabilité horizontale des traitements NLP.
Stanford CoreNLP	Moteur d'extraction (NLP)	Fournit les modules NER, Relation Extraction (KBP) et Entity Linking
Neo4j	BD Graph	Stockage des triplets et interface d'interrogation Cypher
Apache Jena Fuseki	Point de terminaison SPARQL	Hébergement local de Wikidata pour l'extraction ciblée des faits et l'enrichissement contrôlé du graphe

CurEx était divisé en deux grands composants. De manière similaire, l'approche dstlr est organisée en deux phases principales : Extraction et Enrichment (enrichissement)

1. Extraction

L'objectif de la phase d'extraction est de transformer les documents textuels en un graphe brut composé de mentions, relations et liens vers des entités externes.

Entrées :

- Un corpus de documents non structurés (stocké dans Apache Solr).
- Un pipeline d'analyse linguistique (Stanford CoreNLP).

Sortie : Un graphe brut contenant des nœuds Document, des nœuds Mention, des relations entre mentions, des liens mention → entité (Wikidata).

Cette phase comprend trois sous-étapes :

Etape 1: Reconnaissance d'Entités Nommées (NER)

Le système utilise le module NERClassifierCombiner de Stanford CoreNLP. Il détecte 20 types d'entités (personnes, organisations, lieux, …). Pour chaque mention détectée, il enregistre:

- l'id du document
- un id de mention
- le type d'entité
- le texte exact
- les positions (offsets) dans le document

Les occurrences multiples d'une même mention dans un document sont consolidées afin de conserver une traçabilité précise. Cette étape permet de passer du texte brut à une représentation structurée des entités mentionnées.

Etape 2: Extraction de Relations

CoreNLP propose deux types d'extraction : OpenIE (relations ouvertes) et KBP (ensemble fermé de 45 relations prédéfinies). Les auteurs choisissent l'annotateur KBP car les relations prédéfinies facilitent l'alignement ultérieur avec Wikidata.

Chaque relation extraite contient :

- le document_id
- la mention sujet
- le type de relation
- la mention objet

- un score de confiance

Les relations sont modélisées sous forme de nœuds dans Neo4j afin de permettre des requêtes plus efficaces.

Voici un exemple qu'on a dans l'article :

"Facebook is based in Menlo Park" -> relation CITY_OF_HEADQUARTERS entre Facebook et Menlo Park

Etape 3 : Entity Linking

La dernière étape de l'extraction consiste à relier chaque mention à une entité du graphe externe. Le système utilise le WikidictAnnotator de CoreNLP pour associer chaque mention à une page Wikipedia, puis à son identifiant Wikidata correspondant. Même les mentions pour lesquelles aucun lien n'est trouvé sont conservées (valeur nulle), ce qui permet de garder une vision complète des données extraites.

Cette étape est essentielle car elle relie le texte au référentiel externe et rend possible l'enrichissement ultérieur.

Résultats expérimentaux : sur environ 580 000 articles du Washington Post on a 97 millions de triplets extraits, 46,1 millions de mentions, 46,1 millions de liens mention–entité, 4,8 millions de relations entre mentions.

2. Enrichissement

Après la construction du graphe brut, les auteurs l'enrichissent à l'aide de Wikidata. L'objectif est de compléter les entités extraites avec des faits issus du graphe externe afin d'améliorer la qualité et la complétude du graphe final.

Pour commencer, ils définissent manuellement un mapping entre les 45 relations CoreNLP et les propriétés Wikidata.

Par exemple : "headquarters" → propriété Wikidata P159

Pour chaque entité distincte détectée dans le corpus, le système interroge Wikidata via SPARQL. Puis les faits correspondants sont extraits et insérés dans Neo4j.

Seules les entités réellement présentes dans le corpus sont enrichies, ce qui optimise stockage et performances.

3. Exploitation du graphe et applications

Le graphe final est stocké dans Neo4j et interrogeable en langage Cypher. Les auteurs introduisent une approche basée sur l'alignement de sous-graphes (subgraph matching)

Cette approche permet:

- Vérification de faits
- Identification de preuves textuelles pour des faits existants
- Détection d'incohérences
- Découverte de faits manquants
- Génération de données d'entraînement par supervision distante

On distinge 3 cas :

- Relation extraite = fait Wikidata -> confirmation du fait.
- Relation extraite ≠ fait Wikidata -> incohérence (erreur d'extraction ou erreur factuelle).
- Relation extraite sans fait correspondant -> fait manquant dans Wikidata. Toutes ces tâches sont formulées comme des requêtes sur le graphe.

4. Avantages de l'approche

- Pipeline complet et intégré de bout en bout
- Scalabilité démontrée sur corpus réel massif
- Conservation de la provenance des informations
- Intégration directe avec un graphe externe
- Formulation unifiée de plusieurs tâches via requêtes Cypher
- Plateforme open-source

5. Limites de l'approche

- Dépendance au module KBP de CoreNLP (ensemble limité de 45 relations)
- Mapping manuel entre relations et propriétés Wikidata
- Pas d'utilisation de modèles neuronaux récents pour l'extraction
- Dépendance forte à la qualité de Wikidata
- Les erreurs d'extraction peuvent se propager dans le graphe

Automatic Ontology Population from Textual Advertisements: LLM vs. Semantic Approach

L'objectif principal de cet article est de comparer deux approches d'enrichissement automatique d'une ontologie à partir de textes non structurés : une approche sémantique basée sur l'analyse linguistique et ontologique (KOnPoTe) et une approche basée sur un LLM(Claude).

L'enjeu est de déterminer dans quelle mesure un LLM peut remplacer ou compléter une approche sémantique classique pour la population d'ontologies, cad l'ajout automatique d'instances dans une ontologie donnée afin de construire un graphe de connaissances de domaine

Le cas étudié concerne des annonces publicitaires françaises courtes et incomplètes, contenant peu de verbes, peu d'entités nommées et parfois des relations n-aires.

L'article structure son approche autour de deux méthodes comparées.

1. Approche sémantique : KOnPoTe

L'objectif est de peupler automatiquement une ontologie de domaine à partir d'un texte court, en respectant strictement les axiomes ontologiques et la cohérence logique.

Entrées :

- Une ontologie de domaine (classes, propriétés objet, propriétés de données, axiomes, règles SWRL)
- Un doc textuel (annonce publicitaire)
- Un corpus (20 annonces par domaine)

Deux domaines sont étudiés: immobilier (ontologie plus complexe, avec relations n-aires et règles SWRL) et vente de bateaux (ontologie plus simple)

Sorties :

- Une ontologie instanciée (ajout d'individus et d'assertions)
- Un graphe cohérent vérifié par un raisonneur (Pellet)

KOnPoTe repose sur trois grandes étapes :

Etape 1: Comparateur terminologique :un module compare les termes du texte aux entités de l'ontologie (classes, propriétés et individus). Il prend en compte les

variations syntaxiques.

Exemple : détection des classes House, Garden, etc.

Etape 2 : Analyse basée sur le texte

Àpartir des correspondances détectées :

- Instanciation des classes détectées
- Création d'assertions de propriétés
- Association sujet/objet en fonction du contexte de phrase

Seules les assertions ne générant pas d'incohérence sont ajoutées.

Exemple : création d'une instance LivingRoom avec areaInM2 = 50

Etape 3 : Analyse basée sur la connaissance
Cette étape exploite les axiomes ontologiques et les assertions déjà créées. L'objectif est de structurer les relations en forme d'étoile autour de l'individu principal. Elle ajoute uniquement des assertions cohérentes.

Exemple :
liaison entre House et ses Room via contains

Avantages de KOnPoTe :

- Respect strict des axiomes.
- Résultats cohérents (validés par Pellet).
- Bonne gestion des relations n aires.

Limites :

- Génération d'individus redondants
- Difficultés dans les cas textuels complexes
- Ne détecte pas les abréviations non présentes dans l'ontologie

2) Approche LLM: Claude

L'objectif est d'utiliser un modèle génératif pour peupler l'ontologie directement à partir du texte et de l'ontologie fournie en entrée.

Le modèle utilisé est Claude-Instant qui a surtout été choisi pour sa gestion de longs prompts et sa possibilité d'intégrer un fichier joint (l'ontologie)

Entrées :

- Un prompt expliquant la tâche
- Le texte de l'annonce
- L'ontologie complète

Sortie : Ontologie instanciée au format Turtle

L'approche repose sur une utilisation du modèle sans entraînement spécifique pour la tâche de population d'ontologie. Il est exploité sur la base de ses connaissances générales acquises lors de son pré-entraînement.La performance dépend donc beaucoup du prompt engineering.

Les auteurs testent 4 versions successives de prompt engeeniring :

1. Prompt simple -> format non exploitable
2. Ajout format Turtle ->axiomes non respectés
3. Ajout contrainte respect axiomes -> hallucinations
4. Ajout du mot "exclusively" -> limite les hallucinations

Le Prompt 4 est donc retenu pour l'évaluation.

3. Protocole expérimental

Les expérimentations ont été menées sur un corpus composé de 20 annonces par domaine (immobilier et bateaux). Pour chaque domaine un « Gold Standard » (GS) a été construit manuellement. Il s'agit d'une version de l'ontologie initiale enrichie avec les assertions correctes correspondant au contenu des annonces. L'évaluation des deux approches repose sur les mesures suivantes : précision, rappel, F1-score.

Les métriques sont calculées selon deux perspectives :

- Macroscopique : moyenne des performances par annonce (chaque document a le même poids).
- Microscopique : calcul global basé sur l'ensemble des assertions (chaque assertion a le même poids).

La cohérence logique des ontologies générées est vérifiée à l'aide du raisonneur Pellet.

Les résultats montrent que KOnPoTe surpasse Claude dans les deux domaines. KOnPoTe fonctionne mieux avec des ontologies complexes (immobilier). Claude fonctionne mieux avec des ontologies simples (bateaux).

KOnPoTe

Points forts :

- Cohérence
- Respect des axiomes
- Détection correcte des relations n-aires

Points faibles :

- Redondance d'individus

- Confusion dans les cas complexes

Claude

Points forts :

- Pas de redondance
- Détection d'abréviations

Points faibles :

- Incohérences logiques.
- Hallucinations (génération d'assertions pas présentes dans le texte)
- Difficultés avec les relations n-aires

En conclusion l'article montre clairement que les approches sémantiques garantissent la cohérence et l'intégrité logique. Les LLM offrent flexibilité et facilité d'usage mais manquent de contrôle logique. Une approche hybride est suggérée cad utiliser le LLM pour générer des candidats puis filtrer via raisonnement ontologique

A Short Review for Ontology Learning: Stride to Large Language Models Trend

Cet article propose une revue des principales approches d'apprentissage d'ontologies en mettant en évidence l'évolution des méthodes, depuis les approches linguistiques et statistiques traditionnelles jusqu'aux modèles de deep learning puis à l'intégration récente des LLM.

L'article ne propose pas une nouvelle méthode expérimentale mais c'est une analyse de l'état de l'art pour comprendre comment les LLM transforment progressivement le domaine.

Les auteurs rappellent qu'une ontologie est composée de concepts, organisés hiérarchiquement, enrichis par des relations et des axiomes formels. L'apprentissage d'une ontologie consiste donc à extraire et structurer ces éléments à partir de données textuelles.

1. Approches traditionnelles (Shallow Learning)

Les premières méthodes d'apprentissage d'ontologies reposaient sur des techniques linguistiques, statistiques ou logiques.

Les approches linguistiques utilisent des patrons lexico syntaxiques pour identifier des relations hiérarchiques. Les approches statistiques s'appuyaient sur des cooccurrences ou des méthodes de regroupement. Les approches logiques exploitent des mécanismes d'inférence formelle.

Avantages :

- Simplicité
- Rapidité
- Faible besoin en données

Limites :

- Capacité limitée à déduire automatiquement de nouvelles connaissances - Capacité limitée à gérer des corpus de grande taille
- Forte dépendance à l'intervention humaine
- Difficulté à identifier des relations complexes

2. Approches Deep Learning

Avec l'essor des réseaux neuronaux, l'extraction de concepts et de relations est devenu + précise. Des modèles comme BiLSTM, CNN ou BERT ont permis

d'améliorer la détection d'entités et de relations.

Cependant ces approches nécessitent souvent des données annotées et sont coûteuses à entraîner. Elles améliorent la performance d'extraction mais ne garantissent pas une structuration ontologique complète, notamment au niveau des axiomes.

Limites :

- Besoin important de données annotées
- Difficulté d'adaptation à un nouveau domaine sans réentraînement
- Coût computationnel élevé
- Ne garantit pas une structuration logique complète de l'ontologie

3. Intégration des LLM

Les LLM sont présentés comme une nouvelle tendance permettant de dépasser certaines limites des approches précédentes.

Concept Extraction avec LLM

Les modèles de grande taille peuvent être utilisés pour identifier et typer des termes sans entraînement spécifique (zero-shot). Ils sont également capables de générer directement des représentations formelles, par exemple en syntaxe OWL, à partir d'un simple prompt. Grâce à leurs connaissances acquises lors du pré-entraînement, ils peuvent catégoriser des concepts et proposer des relations sémantiques plausibles entre eux.

Extraction de relations hiérarchiques

Les LLM peuvent également être utilisés pour organiser des concepts de manière hiérarchique. Ils sont capables d'identifier des relations du type « est un type de » (par exemple : un chien est un animal) et de structurer ces concepts du plus général au plus spécifique. Cette organisation peut être obtenue à l'aide de requêtes successives (prompting itératif). Certaines études
montrent que, même sans entraînement spécifique, ces modèles peuvent obtenir des résultats comparables à ceux de méthodes supervisées plus traditionnelles.

Extraction de relations non taxonomiques

L'identification de relations non hiérarchiques reste toutefois moins aboutie. Des travaux exploratoires montrent que les LLM peuvent détecter des relations de type part-of ou associatives, notamment dans des domaines spécialisés comme la biomédecine (par exemple avec UMLS). Néanmoins cette utilisation est encore instable et nécessite davantage de validation.

4) Tendances et perspectives

L'article souligne l'émergence d'approches hybrides combinant LLM et raisonnement ontologique à travers des outils comme DeepOnto ou OntoGPT. L'idée n'est plus d'opposer modèles génératifs et méthodes formelles mais de les combiner afin de bénéficier à la fois de la richesse sémantique des LLM et de la rigueur logique des ontologies.

Les auteurs insistent sur la nécessité de développer des benchmarks (jeux de données et protocoles d'évaluation standardisés) adaptés, d'améliorer l'extraction des relations complexes et de maintenir un rôle actif pour l'expertise humaine.

Cet article permet de situer les autres travaux étudiés dans une perspective plus large. Il montre une évolution progressive des méthodes, depuis les approches symboliques vers les modèles neuronaux, puis vers les modèles génératifs de grande échelle. Il met surtout en évidence une orientation vers des approches hybrides qui combinent génération automatique et validation formelle, ce qui rejoint les limites observées dans l'article KOnPoTe vs LLM.

Comparaison des approches

Critère	CurEx	dstlr	KOnPoTe	LLM (Claude)	Shallow OL	Deep OL	LLM-based OL
Objectif	KG multi sources	KG scalable + enrichissement	Population ontologique cohérente	Population ontologique générative	Extraction de concepts via règles	Extraction via réseaux neuronaux	OL via modèles génératifs
Type d'approche	Pipeline modulaire	Pipeline distribué	Sémantique formelle	Générative zero-shot	Symbolique / statistique	Neuronale supervisée	Générative
Cohérence logique	Partielle	Non formelle	Garantie via raisonneur	Non garantie	Faible	Faible	Non garantie
Scalabilité	Bonne	Très bonne	Limitée	Théorique	Limitée	Bonne mais coûteuse	Potentiellement élevée
Limite principale	Relations peu typées	Dépendance outils NLP	Rigidité	Hallucinations	Faible généralisation	Besoin de données annotées	Manque de validation formelle

Les approches étudiées se distinguent d'abord par leur objectif fondamental. CurEx et dstlr visent principalement la construction d'un graphe de connaissances à grande échelle à partir de données textuelles avec une forte dimension de passage à l'échelle. À l'inverse, KOnPoTe et l'approche basée sur LLM se concentrent sur la population d'une ontologie existante en cherchant à instancier correctement des concepts et des relations déjà définis. L'article de revue ne propose pas de système spécifique mais permet de situer ces différentes méthodes dans une évolution plus large du domaine.

Sur le plan méthodologique, 3 grandes familles se dégagent :

- La première regroupe les approches de type pipeline distribué, comme CurEx et dstlr. Elles reposent sur l'intégration de composants NLP existants et mettent l'accent sur la scalabilité, l'automatisation et l'architecture logicielle. L'objectif

est de traiter de grands volumes de données et de produire un graphe exploitable à grande échelle.

- La deuxième correspond aux approches sémantiques formelles, illustrées par KOnPoTe. La priorité est donnée à la cohérence logique et au respect strict des axiomes ontologiques. L'extraction est encadrée par des contraintes formelles et validée par un raisonneur.
- La troisième famille regroupe les approches génératives basées sur les LLM. Elles offrent une grande flexibilité et une capacité à exploiter une connaissance sémantique générale mais avec un controle logique plus faible et un risque d'incohérences.

Une différence centrale entre ces approches concerne précisément la gestion de la cohérence. KOnPoTe garantit la validité logique des résultats grâce au raisonnement ontologique. Les approches basées sur LLM ne fournissent aucune garantie formelle et peuvent produire des assertions incohérentes. CurEx et dstlr occupent une position intermédiaire: ils produisent des graphes exploitables à grande échelle mais sans étape de validation ontologique strict.
En termes de scalabilité, dstlr semble etre l'approche la plus adapté au traitement à grande échelle. KOnPoTe privilégie la précision et la cohérence mais au détriment de la scalabilité. Les LLM sont très scalables puisqu'ils peuvent traiter de grandes quantités de texte sans adaptation spécifique mais leur instabilité impose souvent une étape de validation supplémentaire.

Enfin, chaque approche présente des limites complémentaires. CurEx souffre d'un manque de précision dans le typage des relations. dstlr dépend fortement des outils NLP qu'il intègre. KOnPoTe est contraint par la structure de l'ontologie initiale et peut manquer de souplesse. Les LLM sont problématiques au niveau de la fiabilité logique.

Ces articles mettent en évidence une tension permanente entre quatre dimensions : la scalabilité, la richesse sémantique, la cohérence logique et la flexibilité. Aucune des approches étudiées ne parvient à combiner pleinement ces quatre propriétés, ce qui ouvre la voie à la conception d'une solution hybride plus équilibrée.

Notre solution :

Introduction

Le but de ce projet est de créer un programme capable de lire un texte tout seul et d'en extraire les informations importantes pour les transformer en un **graphe** (une sorte de carte d'idées reliées entre elles).

Au lieu d'avoir un long texte difficile à analyser, mon programme repère les personnages, les lieux et les entreprises, puis il trouve les liens qui existent entre eux.

Pour que ce soit bien organisé, j'ai découpé mon code en plusieurs étapes simples :

1. **Préparation :** On nettoie le texte.
2. **Repérage :** On trouve les noms importants.
3. **Liaison :** On comprend qui fait quoi avec qui.
4. **Dessin :** On crée l'image finale du graphe.

C'est cette structure étape par étape qui rend le système efficace et facile à améliorer.

Architecture générale du système

Le projet est construit comme une **chaîne de traitement (pipeline)**. Chaque module a un rôle précis et passe le relais au suivant. Cette structure permet de tester chaque étape séparément et de s'assurer que les données sont correctes avant de construire le graphe final.

1. Organisation des dossiers

Le projet est découpé en trois dossiers principaux pour bien séparer le code des données :

- **/data :** Contient le fichier texte d'origine (sample.txt).
- **/src :** Regroupe tous les modules Python (le "moteur" du programme).
- **/output :** Stocke les résultats de chaque étape en format JSON, ainsi que le graphe final (format image et base de données).

2. Le Pipeline de traitement

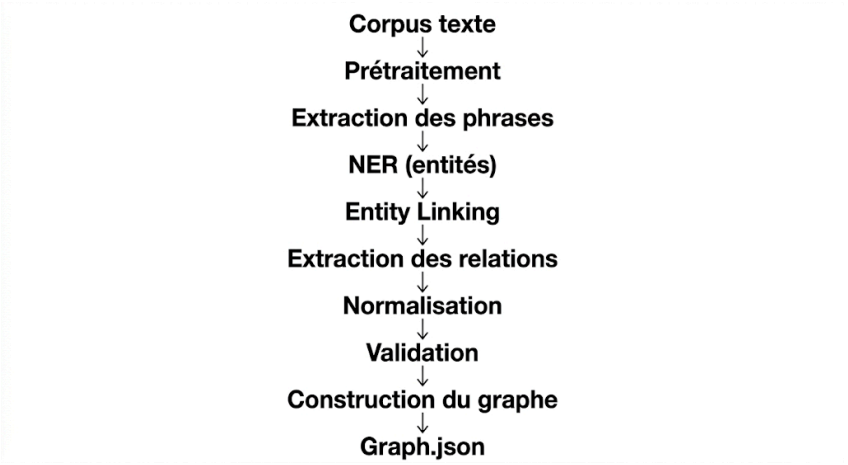
Le cœur du projet (dans le dossier /src) suit ce cheminement :

Étape	Module	Rôle
Prétraitement	Preprocessor	Nettoie le texte et le découpe en phrases.
Extraction (NER)	NERExtractor	Repère les noms (personnes, lieux, orgas).
Liaison	EntityLinker	Attribue un identifiant unique à chaque entité.
Relations	RelationExtractor	Trouve les liens entre les entités dans les phrases.
Normalisation	LLMNormalizer	Utilise l'IA pour uniformiser les noms des relations.
Validation	RelationValidator	Vérifie que les informations extraites sont cohérentes.
Construction	GraphBuilder	Assemble tout en un graphe (JSON, RDF, Cypher).
Visualisation	visualize_graph	Génère une image (.png) pour voir le résultat.

3. Le Chef d'orchestre : main.py

Le fichier main.py est le point d'entrée du programme. C'est lui qui importe tous les modules et les exécute dans le bon ordre. Il s'occupe aussi d'afficher l'avancement dans la console pour que l'utilisateur sache exactement quelle étape est en cours de traitement.

Le pipelines du projet :



Module de prétraitement du corpus (preprocessing)

C'est la première étape du pipeline. Son rôle est de transformer le texte brut (souvent mal rangé) en une liste de phrases propres que l'ordinateur peut analyser facilement.

Ce que fait ce code :

- **Nettoyage :** Il ouvre le fichier texte et enlève tous les retours à la ligne inutiles ou les espaces en trop. On se retrouve avec un texte fluide.
- **Découpage intelligent :** Il utilise une bibliothèque appelée **spacy** (un outil qui comprend la grammaire) pour couper le texte en vraies phrases. Il ne se contente pas de chercher un point, il comprend où une idée s'arrête.
- **Numérotation :** Chaque phrase reçoit un numéro unique (un **sent_id)** pour qu'on puisse la retrouver facilement plus tard dans le projet.

Le résultat : Le module crée un fichier **sentences.json**. Au lieu d'un gros bloc de texte, on a maintenant une liste organisée comme ceci :

> *Phrase 001 : "Jean Dupont habite à Paris." > Phrase 002 : "Il travaille pour Google."*

Module d’extraction des entités nommées (NER)

Une fois que le texte est découpé en phrases, ce module sert de **"détecteur"**. Son rôle est de parcourir chaque phrase pour y trouver les noms propres et les classer par catégories.

Ce que fait ce code :

- **Analyse automatique :** Il utilise à nouveau l'outil **spacy** pour lire les phrases. Il est capable de reconnaître tout seul si un mot est un nom de personne, de ville ou d'organisation.
- **Étiquetage (Labeling) :** Pour chaque mot important trouvé, il lui colle une étiquette (par exemple : **PER** pour une personne, **LOC** pour un lieu, **ORG** pour une entreprise).
- **Précision Chirurgicale :** Le code ne se contente pas de trouver le mot, il note aussi sa position exacte dans la phrase (le début et la fin en nombre de caractères) pour ne pas se tromper si le nom apparaît plusieurs fois.

Le résultat : Le module génère le fichier **entities.json**. Pour chaque détection, on obtient une fiche détaillée :

- **Mention :** "Paris"
- **Catégorie :** Lieu (LOC)
- **Trouvé dans :** Phrase 001

Module d’entity linking

Une fois les noms repérés, il faut savoir de qui ou de quoi on parle exactement. Ce module fait office de **"recherche encyclopédique"** pour éviter les confusions (par exemple, différencier "Orange" le fruit de "Orange" l'entreprise).

Ce que fait ce code :

- **Connexion à Wikidata :** Le programme se connecte à **Wikidata** (la base de données géante derrière Wikipédia) via Internet.
- **Recherche automatique :** Pour chaque nom trouvé (ex: "Paris"), il demande à Wikidata : "C'est quoi ?".
- **Identification unique :** Il récupère un **identifiant unique** (un code qui commence par "Q", comme **Q90** pour Paris), une description et un lien officiel (URL).
- **Gestion du débit :** Le code fait de petites pauses (**time.sleep**) entre chaque recherche pour ne pas saturer le serveur de Wikidata.

Le résultat : Le module crée le fichier **linked_entities.json**. Désormais, chaque nom n'est plus juste un mot, mais une entité bien définie avec sa fiche d'identité numérique :

- **Mention :** "Paris"
- **ID Wikidata :** Q90
- **Description :** Capitale de la France.
- **Lien :** https://www.wikidata.org/wiki/Q90

Module d'extraction des relations

Ce module a pour mission de trouver les liens entre les entités. Il cherche à répondre à la question : "Quel est le rapport entre cette personne et ce lieu (ou cette autre personne) ?". Pour cela, il utilise des **motifs de recherche** (des modèles de phrases).

Ce que fait ce code :

- **Recherche de modèles (Patterns) :** Le programme utilise des expressions régulières (Regex) pour repérer des phrases types, comme par exemple :
  - "*X est né le... à...*"
  - "*X a étudié à...*"
  - "*X est marié avec...*"
- **Découpage intelligent :** Si le texte dit "Il a étudié les maths et la physique", le code est capable de comprendre qu'il y a **deux** sujets d'études différents et crée deux relations distinctes.
- **Mémoire du contexte :** Le code est malin : si une phrase commence par "Il" ou "Elle", il se souvient de la personne dont on parlait juste avant pour ne pas perdre le fil.
- **Nettoyage des étiquettes :** Il retire les petits mots inutiles comme "le", "la" ou "aux" pour ne garder que l'information pure (ex: "France" au lieu de "en France").

Le résultat : Le module crée le fichier **candidate_relations.json**. Il contient des "triplets" (Sujet -> Relation -> Objet) :

- **Sujet :** "Jean Dupont"
- **Relation :** "est né à"
- **Objet :** "Lyon"

- **Preuve :** "Jean Dupont est né le 12 mai à Lyon." (la phrase d'origine pour vérifier).

Module de normalisation des relations

Une fois les relations extraites, elles sont souvent écrites de manières différentes (ex: "est né à", "originaire de", "vient de"). Ce module sert à **uniformiser le langage**. Il transforme ces expressions variées en étiquettes standardisées que l'ordinateur comprend parfaitement.

Ce que fait ce code :

- **Liste autorisée :** Le programme possède une liste fixe de relations officielles, comme **birthDate, bornInCity** ou **marriedTo.** Rien d'autre n'est accepté.
- **Traduction intelligente :** Le code analyse la "relation brute" trouvée précédemment et décide laquelle de la liste officielle lui correspond le mieux.
    - *Exemple :* "est né le" devient automatiquement **birthDate.**
- **Préparation pour l'IA (Prompting) :** Ce qui est très intéressant ici, c'est que le code prépare un "Prompt" (une instruction) destiné à une Intelligence Artificielle (LLM). Il lui donne le sujet, l'objet et la phrase de preuve, puis lui demande de choisir la meilleure étiquette parmi la liste.
- **Justification :** Le module ne se contente pas de changer le nom, il explique pourquoi il a fait ce choix (ex: "indique un lieu de naissance").

Le résultat : Le module crée le fichier **normalized_relations.json**. Les données sont maintenant "propres" et prêtes pour un système informatique pro :

- **Relation brute :** "a travaillé à"
- **Relation normalisée : worksAt**
- **Statut :** "normalized"

Module de validation des relations

C’est l’étape de contrôle qualité. Avant d'ajouter une information dans le graphe final, ce module vérifie qu'elle est cohérente et qu'elle respecte les règles du système (le **schéma**).

Ce que fait ce code :

- **Vérification du type (Le Schéma) :** Le programme possède un dictionnaire de

règles. Par exemple, pour la relation `birthDate`, il vérifie que le sujet est bien une **personne** et que l'objet est bien une **date**. Si on essayait de dire qu'une ville est née d'une personne, le code le rejetterait.

- **Détection intelligente :** Le module contient des listes de mots-clés (pays connus comme "France", mots-clés d'organisations comme "Université") pour vérifier si l'objet de la phrase correspond bien à ce qu'on attend.
- **Vérification de la preuve :** Le code s'assure que la "preuve" (le petit morceau de phrase extrait) est bien présente dans la phrase d'origine. C'est une sécurité pour éviter que l'IA n'invente des faits (hallucinations).
- **Décision (Accepté/Rejeté) :** Chaque relation reçoit un statut. Si elle ne respecte pas les règles, elle est mise de côté avec une explication (ex: "Type de l'objet invalide").

Le résultat : Le module crée le fichier `validated_relations.json`. C'est une liste propre où seules les informations certifiées sont marquées comme **"accepted"**. On y trouve aussi la raison du refus pour les erreurs, ce qui permet de corriger le système.

Module de construction du graphe

Le point final de mon programme n'est pas juste une image, c'est la création d'un fichier au format **RDF** (Resource Description Framework). C'est le langage standard utilisé pour le Web Sémantique.

Pourquoi utiliser le RDF ?

Au lieu de stocker les informations dans un simple tableau, le RDF les organise sous forme de **"Triplets"**. Un triplet, c'est comme une petite phrase logique :

1. **Sujet :** L'entité principale (ex: *Jean Dupont*).
2. **Prédicat :** La relation (ex: *bornInCity*).
3. **Objet :** La destination (ex: *Lyon*).

Ce que mon code permet de faire :

- **Exportation en Turtle (.ttl) :** Mon module GraphBuilder transforme toutes les relations validées en un fichier texte spécialisé. Ce format est lisible aussi bien par un humain que par une machine.
- **Identifiants uniques :** Grâce à l'étape de *Liaison d'Entités*, chaque personnage ou lieu possède son propre lien (URI). Cela permet de relier mon graphe à d'autres bases de données mondiales comme **Wikidata** ou **DBpedia**.

- **Requêtes intelligentes :** Avec ce format, on peut poser des questions complexes au graphe, comme : *"Trouve-moi toutes les personnes nées en France qui ont étudié la physique"*.

En résumé, le passage en RDF transforme une simple liste de phrases en une véritable **base de connaissances structurée et connectée au reste du monde**.

Conclusion

Ce projet m'a permis de mettre en place un **pipeline complet d'extraction de connaissances**, allant du texte brut jusqu'à la création d'un graphe structuré.

Grâce à cette architecture modulaire, le système est capable de :

1. **Comprendre le texte** en identifiant précisément les noms et les lieux (NER & Linking).
2. **Extraire du sens** en repérant les relations entre ces entités (Extraction).
3. **Garantir la qualité** des données grâce à une étape de normalisation par IA (LLM) et une validation rigoureuse (Schéma).

Le résultat final n'est pas seulement une image, mais une véritable base de données au format **RDF**. Ce format standard du Web Sémantique permet à mon graphe d'être "intelligent" : il peut être partagé, relié à des bases mondiales comme Wikidata, et interrogé de manière complexe.

Ce travail montre qu'avec une organisation par étapes (pipeline), on peut transformer une masse d'informations textuelles désordonnées en un réseau de connaissances clair, visuel et exploitable techniquement.

***